Task B4 – Final Engineering Decision (5 Marks, K5)

Run every program yourself and attach screenshots linked to the test they show. Numbers labelled **sandbox** were measured by me on a 2-core cloud machine while preparing this answer, so repeat them on your computer and report your own figures. `<fill>` marks data you must collect.

Honest note: the migration needs MySQL, which I did not have in my sandbox. The program **compiles**, and its validation rules were tested separately (output below). Run it against your own test database to get the failure report and metadata output `<fill>`.

B4.1 – Practical migration strategy (`Migration.java`)

Strategy: (1) read the existing pass file with a buffered reader, (2) validate each line (5 fields, non-empty text, integer route, positive numeric amount, no duplicate pass ID) and collect failures **with line number and reason**, (3) load all valid records with `PassLoader.loadAll` (batches of 1,000 inside **one transaction**; any database error rolls everything back), (4) print the report.

```
import java.io.IOException;
import java.nio.file.*;
import java.sql.*;
import java.util.*;

/** Reads the existing pass file, validates each line, loads valid
rows to MySQL in batches
 *  inside one transaction, and prints a report of the rejected
records. */
public class Migration {
    record Failure(int lineNo, String line, String reason) {}

    static String check(String line) {                    // same
rules as PassValidator
        if (line == null || line.isBlank()) return "empty line";
        String[] f = line.split(",", -1);
        if (f.length != 5) return "expected 5 fields but found " +
f.length;
        for (int i = 0; i < 3; i++) if (f[i].isBlank()) return "field
" + (i + 1) + " is empty";
        try { Integer.parseInt(f[3].trim()); } catch
(NumberFormatException e) { return "route is not an integer"; }
        try { if (Double.parseDouble(f[4].trim()) <= 0) return "amount
must be positive"; }
        catch (NumberFormatException e) { return "amount is not a
number"; }
        return null;
    }
```

```
    public static void main(String[] args) throws IOException,
SQLException {
        Path file = Path.of(args.length > 0 ? args[0] : "passes.csv");
        List<String[]> good = new ArrayList<>();
        List<Failure> bad = new ArrayList<>();
        Set<String> seen = new HashSet<>();
        int lineNo = 0;
        try (var r = Files.newBufferedReader(file)) {
            String line;
            while ((line = r.readLine()) != null) {
                lineNo++;
                String why = check(line);
                String[] f = why == null ? line.split(",", -1) : null;
                if (why == null && !seen.add(f[0])) why = "duplicate
pass ID " + f[0];
                if (why != null) bad.add(new Failure(lineNo, line,
why)); else good.add(f);
            }
        }
        try (Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/buspass_test"
, "root", "password")) {
            int loaded = PassLoader.loadAll(con, good);          //
batches of 1000, one transaction, rollback on error
            System.out.println("Read " + lineNo + " lines: loaded " +
loaded + ", rejected " + bad.size());
            for (Failure f : bad) System.out.println("  line " +
f.lineNo() + ": " + f.reason() + "   [" + f.line() + "]");

            DatabaseMetaData md = con.getMetaData();             //
B4.2
            System.out.println("Product : " +
md.getDatabaseProductName() + " " + md.getDatabaseProductVersion());
            System.out.println("Driver  : " + md.getDriverName() + " "
+ md.getDriverVersion());
            try (ResultSet t = md.getTables("buspass_test", null, "%",
new String[]{"TABLE"})) {
                while (t.next()) System.out.println("Table   : " +
t.getString("TABLE_NAME"));
            }
        }
    }
}
```

(It uses `PassLoader` from Task B3.)

Validation rules tested (sandbox, `Migration.check` on sample lines):

```
P1,R1,Arun,12,4500   -> null  (valid)
P2,R2,Div            -> expected 5 fields but found 3
```

```
P3,R3,Me,12,abc      -> amount is not a number
(empty line)         -> empty line
P5,R5,K,9,-1         -> amount must be positive
```

Expected report format (for `passes.csv` containing the damaged lines):

```
Read 8004 lines: loaded 8000, rejected 4
  line 17: expected 5 fields but found 3   [P0017,192311017,Div]
  line 20: amount is not a number   [P0020,192311020,Meena,12,abc]
  ...
```

Paste your real console output and a `SELECT COUNT(*)` that equals the loaded number `<fill>`.

B4.2 – Inspect the database (`DatabaseMetaData`)

The program prints `getDatabaseProductName()/getDatabaseProductVersion()`, `getDriverName()/getDriverVersion()` and the table names from `getTables(...)`. Example of what it prints: `Product : MySQL 8.x`, `Driver : MySQL Connector/J 8.x`, `Table : bus_pass`. `<fill>` with your output.

Why it is useful when the application is moved to another server: the program can check at start-up that it is talking to the expected product and version (SQL dialects and features differ), that the driver is compatible, and that the required tables exist, and it can report a clear error instead of failing in the middle of work. Diagnostic logs from different servers also show exactly which environment produced a problem, and generic tools can discover tables and columns without hard-coded names.

B4.3 – Production decision: use a database

Requirements at the new scale: 12 campuses; 100× the passes (≈ 8,000 → ≈ 800,000 per year); **several counters issuing passes at the same time**; fast search by pass ID and route; safe against lost or corrupt data.

Requirement	Text file	Java serialization	Database (MySQL via JDBC)
Concurrent counters	Several processes appending to one file can interleave or corrupt lines unless explicit file locking is added; no real support	One file written as a whole object: the last writer overwrites the others (lost updates)	Built in: transactions and row locks; a `PRIMARY KEY` on `pass_id` rejects duplicates atomically
Search by ID at scale	O(n) scan. My sandbox measured ≈ 17.5 ms per search at 100,000 records	Must load the whole object into memory (O(n) memory), then	Index on `pass_id`: **O(log n)** per lookup, independent of file size; an index

Requirement	Text file	Java serialization	Database (MySQL via JDBC)
	and a scan grows linearly, so ≈ 140 ms at 800,000, and with 12 campuses sharing a file every search pays it	O(n) or a map	on `route_no` gives fast route listing
Safety and recovery	A crash can leave a half-written last line; no rollback	A crash mid-write can corrupt the whole file; class changes break old files	ACID transactions: a crash leaves data consistent; backups and replication are standard
Evidence from my measurements	Buffering helped by ≈ 650× (B1.3), but a scan is still O(n) (B1.4) and a map index reduced 1,000 searches from ≈ 24 s to under 1 s but is held in one process's memory (B1 alternative)	Same limits as text, plus versioning issues (A4.2)	Batching and one transaction give fast, all-or-nothing imports (B3); `PreparedStatement` prevents injection (A5/B2)

Decision: use a **shared relational database** with `pass_id` as primary key and indexes on `route_no` (and `reg_no`). Counters insert one pass each through a `PreparedStatement` in auto-commit mode (each insert is atomic), and bulk imports use the batch + transaction method of B1/B3. Keep text files only for export and migration.

Alternative concept and complexity

Each campus keeps its own serialized file instead of one shared database:

- **Searching all passes:** a pass could be in any of the 12 files, so a search must load and scan all of them: time **O(N)** in the total number of passes N (each campus file O(N/12) if load is balanced, but all 12 must be read). A database with an index answers in **O(log N)**.
- **Space:** each campus loads its whole file into memory: O(N/12) per campus process, but a global search or report needs all 12 to be loaded or sent to one place: O(N). There is also duplicated storage if a pass must be known to several campuses. The database keeps one copy on the server plus the index (O(N) extra on disk).

- **Concurrent counters:** within one campus several counters would need to share one in-memory object and synchronise (`synchronized`, locks) and write the whole file after each change (or risk losing updates) — workable for one process, but counters on different machines cannot share the object at all. Across campuses nothing keeps the data consistent (the same pass ID can be issued twice). With a shared database the concurrency control, duplicate prevention and consistency are the job of the database's transactions and unique key.